

기술적 성취와 문제 해결 역량

800자

- 비용 60% 절감과 성능 10배 개선: 아키텍처로 증명하는 최적화 역량

안녕하세요. :)

인터랙티브한 사용자 경험을 성능과 구조로 설계하는 백엔드 개발자,

풀스택 개발자를 지향하는 천규진입니다.

저는 백엔드 개발에서 특히 API 설계와 성능·비용 최적화에 깊은 관심을 두고 있으며,

이는 캡스톤 프로젝트인 '여기물까'를 통해 구체화되었습니다.

해당 프로젝트는 온라인 쇼핑몰의 신뢰도를 AI·머신러닝으로 분석해 소비자 피해를 예방하는 플랫폼으로,

학과 대상과 전교 우수상을 수상하며 그 가치를 인정받았습니다.

프로젝트 초기에는 Claude 3.5 Sonnet API를 리뷰 단위로 호출하면서 3초 이상의 응답 지연과

과도한 비용 문제가 발생했습니다. 원인을 분석한 결과, 중복 호출과 단건 처리 구조가

핵심 문제임을 파악했습니다. 이 과정에서 Cursor의 코드베이스 분석 기능을 활용해

기존 로직의 병목 지점을 빠르게 식별했고, LLM과 소통하며 최적의 배치 사이즈에 대한 트레이드오프를 검토했습니다.

이를 통해 리뷰 요청을 15개 단위 배치 처리로 전환하고,

5분 캐싱 전략을 도입해 중복 호출을 제거했습니다.

그 결과 API 비용을 60% 절감하고 응답 속도를 3초에서 0.5초로 단축할 수 있었습니다.

또한 PostgreSQL Full-Text Search와 인덱싱을 적용해 검색 속도를 기존 대비 10배 이상 개선했으며,

Node.js와 Python 서버를 분리해 각 언어의 장점을 살리는 마이크로서비스 구조를 설계했습니다.

이 경험을 통해 API 설계는 단순한 엔드포인트 구현이 아니라

성능, 비용, 운영을 함께 고려한 아키텍처 설계라는 것을 깊이 체감했습니다.

협업 마인드 및 성장 태도

605자

- 시스템 전체를 조망하는 시야와 우선순위 중심의 문제 해결

저는 또한 포트폴리오 웹사이트를 데이터 기반 구조로 직접 설계하여 유지보수성을 높였으며,

이미지 최적화와 지연 로딩을 통해 모바일 성능을 개선하는 등 프론트엔드 영역까지 경험을 확장했습니다.

이렇게 프론트엔드와 백엔드를 모두 경험하며 전체 시스템의 흐름을 이해하게 되었고,

이는 백엔드 개발자로서 프론트엔드의 요구사항과 제약 사항을 미리 고려하여

더 효율적인 API를 설계할 수 있는 역량으로 이어졌습니다.

저의 가장 큰 장점은 문제를 발견하면 단순히 해결하는 것에 그치지 않고,

그 원인과 기술적 트레이드오프를 면밀히 분석해 끝까지 완수하는 태도입니다.

때로는 이러한 완벽주의로 인해 초기 단계에서 과도한 최적화를 시도하며 시행착오를 겪기도 했습니다.

하지만 이를 계기로 실제 서비스 운영 환경에서는 MVP(Minimum Viable Product)

우선 개발과 비즈니스 가치에 따른 우선순위 관리의 중요성을 명확히 배울 수 있었습니다.

앞으로도 성능과 확장성을 함께 고려하며, 기술적 성장이 서비스의 핵심 가치로

안정적으로 연결될 수 있도록 끊임없이 고민하는 풀스택 개발자로 성장하고 싶습니다.

긴 글 읽어주셔서 대단히 감사합니다. :)